

Aula 4

NLP: Vetorização de Texto e Word Embeddings

Tópicos Avançados em Inteligência Artificial · UFABC

Parte 1 — Por que NLP?

Roteiro da aula

Parte 1 — Por que NLP? Motivação, aplicações e arco histórico

Parte 2 — Pré-processamento de Texto Pipeline
Standardize → Tokenize → Index → Encode

Parte 3 — Bag of Words Count encoding, multi-hot, limitações

Parte 4 — Word Embeddings Problema das one-hot, Word2Vec, GloVe

Parte 5 — Embeddings em Código `nn.Embedding` (PyTorch), `Embedding` (Keras)

Parte 6 — Aplicação Classificação de texto com embeddings

Por que NLP?

O conhecimento humano está em linguagem natural

- A Internet é, em sua maioria, **texto**
- Comunicação humana e produção cultural são **texto**
- Documentos corporativos, contratos, registros médicos são **texto**

Imagine um sistema que pudesse ler e "compreender" tudo isso automaticamente — extraindo padrões, classificando, respondendo perguntas, gerando conteúdo.

Aplicações em produção

Classificação

Sentimento, intenção, roteamento

Extração

Financeiros, dados de formulários

Sumarização

Abstracts, bullet points, títulos

Geração

Emails, relatórios, código

Q&A / RAG

Chatbots, busca semântica

Tradução

Multilingual, code-switching

Arco histórico do NLP



Regras

até 1990

- Gramáticas formais
- Analisadores sintáticos manuais
- Desempenho limitado à escala das regras



Estatístico / ML

1990 – 2013

- Bag of Words + SVM
- Modelos de Markov ocultos
- Regressão logística sobre n-gramas



Redes Neurais

2014 – 2017

- RNNs e LSTMs
- Word2Vec, GloVe
- Seq2seq com atenção



Transformers

2017 – hoje

- "Attention is All You Need"
- BERT, GPT, T5, LLaMA
- LLMs de propósito geral

"Every time I fire a linguist, the performance of the speech recognizer goes up." — Frederick Jelinek, IBM

Visão geral: NLP como regressão

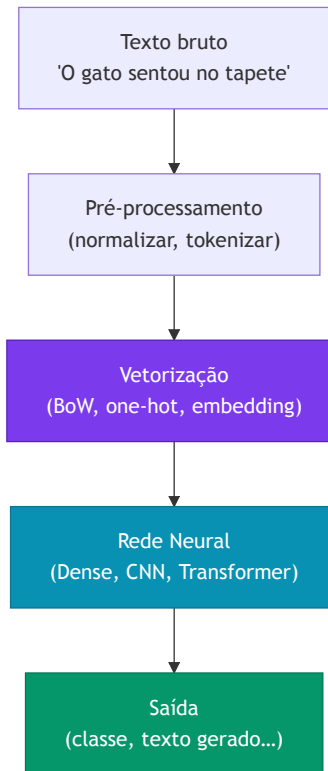
No fundo, é uma regressão sofisticada

$$\hat{y} = f(\mathbf{x}, \theta)$$

- $\mathbf{x}$ = texto de entrada (sequência de tokens)
- $\hat{y}$ = texto, rótulo, número, ...
- θ = pesos da rede neural
- f = arquitetura profunda (CNN, RNN, Transformer...)

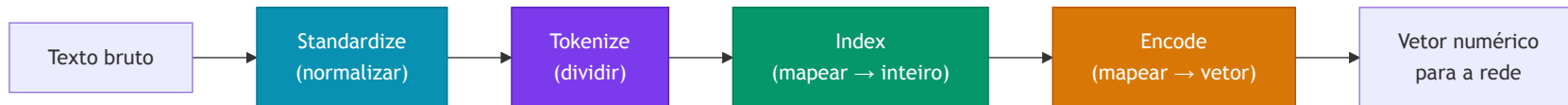
Questão central desta aula:

Como representar $\mathbf{x}$ (texto) como vetor numérico de forma que a rede possa aprender?



Parte 2 — Pré-processamento de Texto

Pipeline de pré-processamento



Standardize

- Minúsculas
- Remove pontuação/acentos
- *Stop words* (às vezes)
- Stemming/lemmatização (às vezes)

Tokenize

- Divide em tokens
- Padrão: por espaço em branco
- Alternativa: subpalavras (BPE)
- N-gramas (bigrama, trigrama...)

Index

- Atribui inteiro único a cada token
- Forma o **vocabulário** $\mathcal{V}$
- Token especial `<UNK>` para palavras fora do vocabulário

Encode

- Mapeia inteiro → vetor
- Mais simples: one-hot
- Melhor: word embedding

Exemplo de pré-processamento

Texto original:

```
"Hola! What do you picture when you think of traveling to Mexico? Sipping a real margarita while soaking up the sun on a laid-back beach in Puerto Vallarta?"
```

Após standardize (minúsculas, remove pontuação, stop words, stemma):

```
`hola picture think travel mexico sip real margarita soak sun  
laidback beach puerto vallarta`
```

Após tokenize (por espaço):

```
`["hola", "picture", "think", "travel", "mexico", "sip", "real",  
"margarita", "soak", "sun", "laidback", "beach", "puerto",  
"vallarta"]`
```

Após index (vocabulário com 50 k tokens):

```
`[4231, 1807, 2943, 8821, 9012, 3344, 771, 18432, 6621, 488, 22301, 993, 19034, 19035]`
```

One-hot encoding

Ideia: cada token do vocabulário recebe um vetor com 1 na sua posição e 0 em todas as outras.

Vocabulário ($|\mathcal{V}| = 5$): gato, cão, peixe, pássaro, rato

gato → [1, 0, 0, 0, 0]

cão → [0, 1, 0, 0, 0]

peixe → [0, 0, 1, 0, 0]

pássaro → [0, 0, 0, 1, 0]

rato → [0, 0, 0, 0, 1]

- Dimensão do vetor = $|\mathcal{V}|$ (tamanho do vocabulário)
- Token especial `<UNK>` (índice 0) para palavras fora do vocabulário
- Representação **esparsa** — apenas um elemento não-nulo

Problema: para $|\mathcal{V}| = 50,000$, cada token vira um vetor de 50 mil dimensões.

- ❌ **Dimensionalidade** altíssima → muitos parâmetros, overfitting
- ❌ **Sem noção de similaridade** — distância entre quaisquer dois vetores one-hot é sempre $\sqrt{2}$, independentemente do significado

```
from sklearn.preprocessing import LabelBinarizer

vocab = ["gato", "cão", "peixe", "pássaro", "rato"]
lb = LabelBinarizer()
lb.fit(vocab)
print(lb.transform(["gato", "peixe"]))
# [[1 0 0 0 0]
#   [0 0 1 0 0]]
```

Parte 3 — Bag of Words

Do vetor por token ao vetor por documento

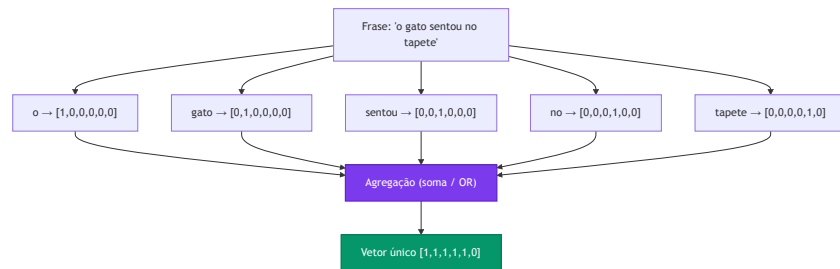
O problema: frases têm comprimentos diferentes.

Uma frase com 5 tokens gera uma matriz $5 \times |\mathcal{V}|$.

Uma frase com 20 tokens gera $20 \times |\mathcal{V}|$.

Redes densas precisam de **entrada de tamanho fixo**.

Solução: agregar (*pooling*) os vetores one-hot de todos os tokens em um único vetor de tamanho $|\mathcal{V}|$.



Bag of Words — duas variantes

Count encoding (soma dos vetores one-hot)

Conta quantas vezes cada token aparece.

```
"o gato sentou no tapete"  
"o tapete era do gato"  
  
vocab = [o, gato, sentou, no, tapete, era, do]  
  
Frase 1: [1, 1, 1, 1, 1, 0, 0]  
Frase 2: [1, 1, 0, 0, 1, 1, 1]
```

Se "gato" aparecer 3×: componente = 3.

Multi-hot encoding (OR dos vetores one-hot)

Indica apenas se o token está presente (0 ou 1).

```
"o gato gato gato sentou"  
  
Count:      [1, 3, 1, 0, 0, 0, 0]  
Multi-hot: [1, 1, 1, 0, 0, 0, 0]
```

Multi-hot ignora frequência; útil quando presença é mais relevante que contagem.

Ambas são formas de **Bag of Words** — o nome vem do fato de se tratar o texto como um "saco" de palavras sem ordem.

N-gramas — capturando contexto local

Unigrama: 1 token por vez (BoW padrão)

Bigrama: pares de tokens consecutivos

Trigrama: trios de tokens consecutivos

```
Frase: "o gato sentou no tapete"
```

```
Unigramas: ["o", "gato", "sentou", "no", "tapete"]
```

```
Bigramas: ["o gato", "gato sentou",  
           "sentou no", "no tapete"]
```

```
Trigramas: ["o gato sentou",  
            "gato sentou no",  
            "sentou no tapete"]
```

Por que usar n-gramas?

- ✓ Preserva algum contexto local
- ✓ "não bom" ≠ "bom" (bigrama captura a negação)
- ✓ Simples, funciona bem em classificação de texto

- ✗ Vocabulário cresce explosivamente: $|\mathcal{V}|^n$
- ✗ Ainda ignora dependências de longo alcance
- ✗ "bom mas não ótimo" exigiria um 4-grama

```
from sklearn.feature_extraction.text import CountVectorizer  
cv = CountVectorizer(ngram_range=(1, 2))  
X = cv.fit_transform(["o gato sentou no tapete"])
```

Limitações do Bag of Words

❌ Perde a ordem

"O cão mordeu o homem"
e

"O homem mordeu o cão"

têm o **mesmo** vetor BoW.

❌ Alta dimensionalidade

$|\mathcal{V}|$ pode ser 50 k–500 k.

Cada entrada tem essa dimensão,
independente do tamanho da frase.

Muitos parâmetros → overfitting,
lento.

❌ Sem semântica

"filme" e "cinema" são equidistantes
entre si e de "banana".

BoW não captura que são palavras
relacionadas.

Resumo: BoW é uma linha de base útil e simples, mas para tarefas que exigem compreensão semântica ou dependências de longo alcance precisamos de algo melhor — **Word Embeddings**.

Parte 4 — Word Embeddings

O problema das representações one-hot

Problema 1: Alta dimensionalidade

Vocabulário de 50 k palavras → vetor de 50 k dimensões.

Maioria dos elementos é zero (**esparso**).

Problema 2: Sem distância semântica

```
"filme" → [1, 0, 0, 0, 0, ...]
```

```
"cinema" → [0, 1, 0, 0, 0, ...]
```

```
"banana" → [0, 0, 1, 0, 0, ...]
```

```
cosine("filme", "cinema") = 0
```

```
cosine("filme", "banana") = 0
```

```
# Todas as palavras são igualmente "distantes"!
```

A solução ideal

Vetores **densos e compactos** (dimensão 50–300) onde a **distância geométrica** reflita a **distância semântica**.

```
"filme" ≈ [0.2, 0.8, -0.3, ...] # compacto
```

```
"cinema" ≈ [0.19, 0.79, -0.28, ...] # próximo de "filme"
```

```
"banana" ≈ [-0.6, 0.1, 0.9, ...] # longe de ambos
```

```
cosine("filme", "cinema") ≈ 0.97 ✓
```

```
cosine("filme", "banana") ≈ 0.12 ✓
```

A intuição: "você conhece uma palavra pela companhia que ela faz"

"You shall know a word by the company it keeps."

— John Firth, linguista (1957)

Contextos similares → palavras relacionadas

"A atuação no _____ foi incrível."

- | | |
|------------|---|
| → filme | ✓ |
| → cinema | ✓ |
| → musical | ✓ |
| → caminhão | ✗ |
| → banana | ✗ |

Como "filme", "cinema" e "musical" aparecem nos **mesmos contextos**, inferimos que são semanticamente relacionados.

Ideia chave

Contar com que frequência dois tokens co-ocorrem nos mesmos contextos e aprender vetores que **aproximem** essa matriz de co-ocorrência.

Essa é a base de dois algoritmos muito influentes:

- **Word2Vec** (Mikolov et al., 2013) — predição local
- **GloVe** (Pennington et al., 2014) — co-ocorrência global

Word2Vec — dois modelos

CBOW (Continuous Bag of Words)

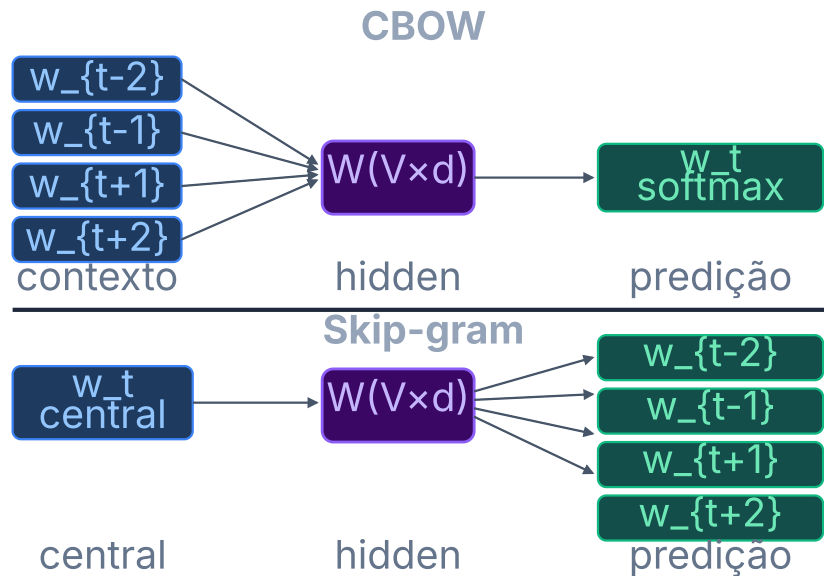
Dado o contexto, prediz a palavra central.

Contexto → Palavra central
["A", "atuação", "foi", "incrível"] → "no ___"

Skip-gram

Dada a palavra central, prediz as palavras de contexto.

Palavra central → Contexto
"filme" → ["A", "atuação", "foi", "incrível"]



💡 **Negative Sampling** — evita softmax sobre $|V|$: classifica (w, ctx) como par real ou aleatório usando $k \approx 5-20$ negativos por passo $\rightarrow O(k)$ em vez de $O(|V|)$

Word2Vec — Negative Sampling

Problema: softmax sobre 50 k tokens a cada passo é custoso — $O(|\mathcal{V}|)$.

Solução — Negative Sampling:

Em vez de normalizar sobre todo o vocabulário, treina um **classificador binário**:

$$P(D=1 \mid w, c) = \sigma(\mathbf{v}_c^\top \mathbf{v}_w)$$

"Este par (palavra, contexto) veio do corpus real?"

```
Par positivo: ("filme", "atuação") → label 1
Pares negativos (k amostras aleatórias):
  ("filme", "mesa") → label 0
  ("filme", "nuvem") → label 0
  ...
```

Objetivo com negative sampling:

$$\mathcal{L} = \log \sigma(\mathbf{v}_c^\top \mathbf{v}_w) + \sum_{i=1}^k \mathbb{E}_{w_i \sim P_n} [\log \sigma(-\mathbf{v}_{w_i}^\top \mathbf{v}_w)]$$

- ✓ Complexidade por passo: $O(k)$ com $k \ll |\mathcal{V}|$ (tipicamente $k = 5-20$)
- ✓ Empiricamente produz embeddings de qualidade similar ao softmax completo

Palavras mais frequentes têm maior probabilidade de serem amostradas como negativas (distribuição $P_n(w) \propto f(w)^{3/4}$).

GloVe — Global Vectors

Intuição: Word2Vec usa janelas locais. GloVe usa **co-ocorrências globais** do corpus.

Passo 1: Contar X_{ij} = quantas vezes a palavra j aparece no contexto da palavra i em todo o corpus.

	filme	cinema	banana	...
filme	[-, 8432, 12, ...]			
cinema	[8432, -, 9, ...]			
banana	[12, 9, -, ...]			

"filme" e "cinema" co-ocorrem muito; ambos pouco com "banana".

Passo 2: Aprender vetores que aproximem a matriz de co-ocorrência.

Modelo log-bilinear:

$$\log X_{ij} = \mathbf{w}_i^\top \mathbf{w}_j + b_i + b_j$$

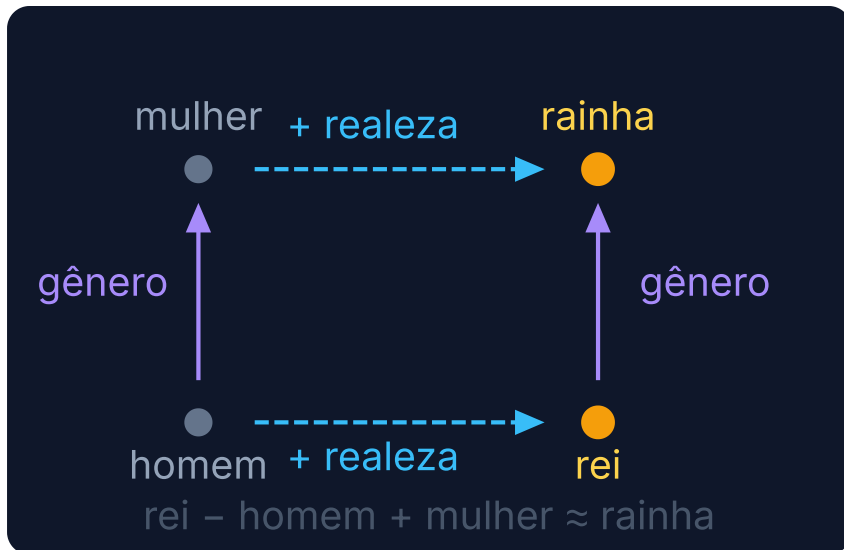
Objetivo (mínimos quadrados ponderados):

$$\mathcal{L} = \sum_{i,j} f(X_{ij}) (\mathbf{w}_i^\top \mathbf{w}_j + b_i + b_j - \log X_{ij})^2$$

onde $f(X_{ij})$ é uma função de ponderação que evita que pares muito frequentes dominem o treinamento.

Ao final, descartam-se os biases b e usa-se apenas $\mathbf{w}$.

Propriedades geométricas dos embeddings



Projeção 2D — cada dimensão captura um conceito latente que emerge do treinamento.

Tipos de relações que emergem

Gênero

rei → rainha · ator → atriz · homem → mulher

País → Capital

França → Paris · Brasil → Brasília · Japão → Tóquio

Superlativo / Grau

bom → melhor → ótimo · grande → maior → máximo

Tempo verbal

correr → correu · cantar → cantou · ir → foi

Antônimos

quente ↔ frio · rápido ↔ lento · grande ↔ pequeno

⚠ Esses padrões emergem do corpus — e também refletem seus **vieses** (Wikipedia, Common Crawl...).

FastText — Embeddings de Subpalavras

Problema do Word2Vec/GloVe: cada palavra tem um único vetor — palavras fora do vocabulário (OOV) ficam sem representação.

Ideia do FastText (Bojanowski et al., 2017):
em vez de aprender um vetor por palavra, aprende vetores para **n-gramas de caracteres**.

```
"where" → n-gramas (n=3):  
  <wh, whe, her, ere, re>  
  + o token inteiro <where>
```

```
vetor("where") =  $\sum$  vetores dos n-gramas
```

- ✅ **OOV:** "whereabouts" nunca visto → combina n-gramas conhecidos
- ✅ **Morfologia:** "correr", "correndo", "corrida" compartilham n-gramas
- ✅ **Ruído/typos:** "teh" $\approx$ "the" via n-gramas sobrepostos

Comparação

	Word2Vec / GloVe	FastText
Unidade	palavra	n-grama de char
OOV	✗ sem vetor	✅ compõe
Morfologia	✗ ignora	✅ captura
Velocidade	✅ rápido	⚠ mais lento
Tamanho	✅ menor	⚠ maior

Uso prático: FastText é especialmente útil em idiomas com **morfologia rica** (português, alemão, finlandês) e em textos com erros ortográficos (redes sociais). Vetores pré-treinados disponíveis para 157 idiomas em fasttext.cc.

Embeddings estáticos vs contextuais

Embeddings estáticos

(Word2Vec, GloVe, FastText)

- Um vetor fixo por palavra, independente do contexto
- Rápidos de computar, baixo custo de memória
- Bons para tarefas de classificação simples

❌ "banco" (financeiro) tem o mesmo vetor que "banco" (assento)

Embeddings contextuais

(ELMo, BERT, GPT)

- Vetor depende de todos os outros tokens da frase
- Captura polissemia e dependências de longa distância
- Base dos LLMs modernos

✅ "banco" gera vetores diferentes em cada contexto — calculados por um **Transformer** (Aula 6!)

Esta aula cobre embeddings estáticos. A **Aula 5** cobre RNNs/LSTMs e a **Aula 6** Transformers e embeddings contextuais.

Parte 5 — Embeddings em Código

nn.Embedding — PyTorch

O que é: uma tabela de consulta — mapeia índices inteiros para vetores densos.

```
import torch
import torch.nn as nn

# vocab_size = tamanho do vocabulário
# embed_dim = dimensão do embedding
embed = nn.Embedding(
    num_embeddings=10_000, # |V|
    embedding_dim=128      # d
)

# Índices de tokens para um batch de 2 frases
# (padding com zeros até max_len=5)
tokens = torch.tensor([
    [42, 871, 3, 0, 0],
    [7, 1024, 55, 810, 2]
]) # (batch=2, seq_len=5)

out = embed(tokens) # (2, 5, 128)
print(out.shape)    # torch.Size([2, 5, 128])
```

Carregando pesos pré-treinados (GloVe)

```
import numpy as np

# Supõe glove_matrix: (|V|, d) pré-carregada
embed = nn.Embedding(
    num_embeddings=vocab_size,
    embedding_dim=100
)
embed.weight = nn.Parameter(
    torch.tensor(glove_matrix, dtype=torch.float32)
)

# Opcional: congelar pesos durante fine-tuning
embed.weight.requires_grad = False
```

`nn.Embedding` é equivalente a uma camada `nn.Linear` sem bias, mas com lookup $O(1)$ — não faz multiplicação matricial por frases inteiras.

Camada Embedding — Keras

Pipeline completo com Keras

```
import tensorflow as tf
from tensorflow import keras

# 1. Vetorização (STI)
vectorizer = keras.layers.TextVectorization(
    max_tokens=10_000, # |V|
    output_mode='int', # retorna índices
    output_sequence_length=64 # max_len
)
vectorizer.adapt(train_texts) # aprende vocabulário

# 2. Embedding
embed = keras.layers.Embedding(
    input_dim=10_000, # |V|
    output_dim=128, # d
    mask_zero=True # ignora padding
)
```

Modelo completo

```
inp = keras.Input(shape=(1,), dtype='string')
x = vectorizer(inp) # (batch, 64)
x = embed(x) # (batch, 64, 128)
x = keras.layers.GlobalAveragePooling1D()(x)
# média sobre seq_len → (batch, 128)
x = keras.layers.Dense(64, activation='relu')(x)
out = keras.layers.Dense(num_classes,
    activation='softmax')(x)

model = keras.Model(inp, out)
model.compile(
    loss='sparse_categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)
```

Global Average Pooling — por que funciona?

Após o `Embedding`, cada frase é uma **matriz ($T \times d$)**.

Precisamos de um **vetor fixo** para a camada densa.

```
"o gato sentou" → Embedding
[[0.2, -0.1, 0.8, ...], ← "o"
 [0.7, 0.3, 0.1, ...], ← "gato"
 [0.4, -0.2, 0.6, ...]] ← "sentou"   shape: (T, d)
```

```
GAP → média ao longo de T (dim=1)
→ [0.43, 0.0, 0.5, ...]               shape: (d,)
```

Por que funciona? GAP é equivalente a um **BoW semântico**: cada palavra contribui com seu vetor de significado, e a média captura o "conteúdo médio" da frase. Ao contrário do BoW clássico (onde "filme" e "cinema" são vetores ortogonais), aqui palavras semanticamente próximas contribuem com vetores próximos — o resultado agrega semântica, não apenas contagem.

Alternativas ao GlobalAveragePooling

Flatten — concatena todos os vetores em ($T \times d$), requer comprimento fixo.

GlobalMaxPooling1D — máximo por dimensão, preserva ativações mais fortes.

[CLS] token / atenção — estratégia dos Transformers (Aula 6). BERT usa `[CLS]`.

Limitação comum a todos: nenhuma dessas estratégias preserva a **ordem** dos tokens. Para isso, precisamos de RNNs ou Transformers.

Parte 6 — Aplicação: Classificação de Texto

Classificação de gênero musical

Arquitetura BoW + Embedding

Tarefa: dado um trecho de letra, classificar em hip-hop, pop ou rock.

*"I grew up on the crime side, the New York Times
side... Had secondhands, Mom's bounced on old man..."* →
hip-hop 🎤

*"I walked through the door with you But something
about it felt like home somehow..."* → **pop** 🎵

Código completo — PyTorch

```
import torch, torch.nn as nn, torch.optim as optim

class TextClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, num_classes):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.dropout = nn.Dropout(0.3)
        self.fc1 = nn.Linear(embed_dim, 64)
        self.fc2 = nn.Linear(64, num_classes)

    def forward(self, x):          # x: (batch, seq_len)
        x = self.embed(x)          # (batch, seq_len, embed_dim)
        x = x.mean(dim=1)          # GAP: (batch, embed_dim)
        x = self.dropout(x)
        x = torch.relu(self.fc1(x))
        return self.fc2(x)         # (batch, num_classes)

model = TextClassifier(vocab_size=10_000, embed_dim=64, num_classes=3)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(10):
    for tokens, labels in train_loader:
        logits = model(tokens)
```

Comparação: BoW vs Embeddings

	BoW (count/multi-hot)	Embedding + GAP
Representação	Esparsa ($ V $ -dimensional)	Densa (d -dim, $d \ll V $)
Semântica	❌ Sem distância semântica	✅ Palavras similares são próximas
Ordem	❌ Ignorada	❌ Ignorada (com GAP)
Parâmetros	Nenhum (engenharia)	$ V \times d$ (aprendidos)
Transferência	Só com embeddings externos	✅ Pré-treino em corpus grande
Custo	Baixo	Moderado

Para capturar ordem e dependências longas → arquiteturas sequenciais: RNNs e LSTMs (**Aula 5**), e finalmente Transformers (**Aula 6**).

Recapitulando

Pré-processamento

- Pipeline $S \rightarrow T \rightarrow I \rightarrow E$
- Standardização, tokenização, vocabulário
- One-hot encoding: esparsos, sem semântica

Bag of Words

- Count e multi-hot encoding
- N-gramas para contexto local
- Limitações: ordem perdida, alta dimensão

Word Embeddings

- Word2Vec (skip-gram, negative sampling)
- GloVe (co-ocorrência global, mínimos quadrados)
- Vetores densos com distância semântica

Em código

- `nn.Embedding` (PyTorch)
- `Embedding` + `TextVectorization` (Keras)
- `GlobalAveragePooling1D` como BoW ponderado

Limitações restantes

- Sem ordem com GAP
- Embeddings estáticos: polissemia
- → Precisamos de Transformers!

Aula 5 — RNNs e LSTMs

- RNNs: estado oculto e BPTT
- Problema do gradiente vanishing
- LSTMs e GRUs como solução

Próxima aula

Aula 5 — RNNs e LSTMs

- Estado oculto e processamento sequencial
- BPTT e o problema do gradiente
- LSTM: células de memória e gates
- GRU: variante mais eficiente
- Aplicações: classificação e seq2seq

Para esta semana

- Notebook `lec04-embeddings.ipynb` :
 - Pipeline de pré-processamento com Keras/PyTorch
 - Treinar embeddings do zero vs GloVe pré-treinado
 - Classificação de gênero musical
 - Visualização de embeddings com t-SNE

Obrigado! Perguntas?

CCM-109 · Deep Learning · UFABC

Adaptado de MIT 15.773 Hands-on Deep Learning (Farias & Ramakrishnan, 2024)